

# Graphs: Definitions, Representation, Traversals

*ITCS 4114: Real World Algorithms*

Kalpathi Subramanian

Computer Science, UNC Charlotte

*Last Modified: Feb., 2026*

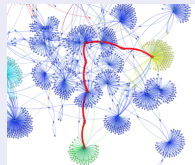
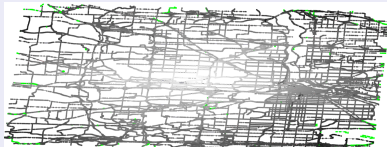
# Graphs: Definitions, Representation, Traversals

## Learning Objectives

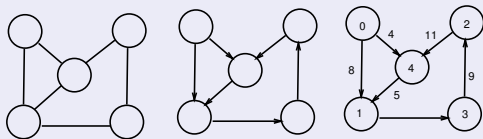
- ♣ Name some applications of graphs in the real world
- ♣ Distinguish between the adjacent matrix and adjacent list based graph representations
- ♣ Given a graph operation, state the computational complexity of the operation in both graph representations.
- ♣ Given a simple graph and a starting vertex, enumerate the nodes visited by a BFS based traversal order.
- ♣ Given a simple graph and a starting vertex, enumerate the nodes visited by a DFS based traversal order.
- ♣ Explain the difference between the BFS and DFS traversal algorithms
- ♣ Given a simple graph, apply the DFS algorithm to determine its connected components.
- ♣ Given a simple graph, apply the BFS algorithm to determine its connected components.

# Graph Applications

- ◇ Communication Networks (Internet)
- ◇ Social networks (Facebook, Twitter).
- ◇ Geographic Maps :Road networks, routes, distances, eg. OpenStreetMap
- ◇ Transportation Networks (air, rail, sea)
- ◇ Actor-Movies
- ◇ Citation networks (authors, publications)
- ◇ Infectious disease spread

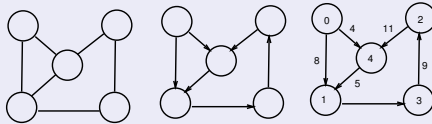


# Definitions and Terminology



- ◇ **Graph:**  $G = (V, E)$ , set of vertices  $V$ , and a set of edges  $E$  connecting pairs of vertices.
- ◇ **Sparse, Dense** graphs.
- ◇ **Complete Graph:** Includes all possible edges.
- ◇ **Digraph:** Directed Graph, where edges are directed.
- ◇ **Undirected Graph:** Edges are undirected.
- ◇ **Adjacency, Neighbors:** vertices sharing an edge
- ◇ **Incidence:** Edge  $(u, v)$  is incident on vertices  $u$  and  $v$ .
- ◇ **Vertex, Edge Data:** Vertices, edges can have data (weight) - application dependent: cost, distance, similarity, etc..

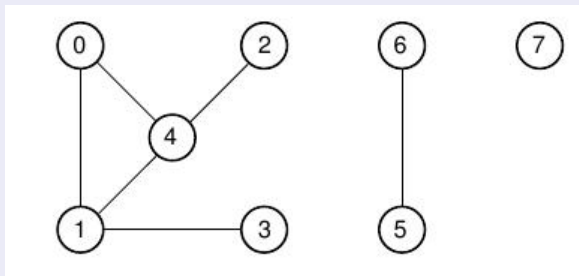
## Definitions (Contd)



- ◇ **Path:** Sequence of vertices  $v_1, v_2, \dots, v_n$  connected by edges.
- ◇ **Simple Path:** All vertices are distinct.
- ◇ **Cycle:** Path of length 3 or more connecting some vertex to itself.
- ◇ **Acyclic:** Graph with no cycles.
- ◇ **DAG:** Directed Acyclic Graph.
- ◇ **Simple Cycle:** Path is simple.
- ◇ **Subgraph:** A graph containing a subset of graph G's vertices and some edges of G.
- ◇ **Connected:** Graph has at least one path from any vertex to any other.
- ◇ **Connected Components:** Maximally connected subgraphs of an undirected graph.

# Graph: Connected Components

An example graph with 3 connected components

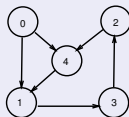


**Note:** Components can be disconnected, but are **still part of the same graph**

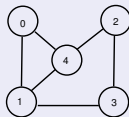
# Graph Representations

Two commonly used representations: **Adjacency Matrix**, **Adjacency List**

## Adjacency Matrix



|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 |   | 1 |   |   | 1 |
| 1 |   |   |   | 1 |   |
| 2 |   |   |   |   | 1 |
| 3 |   |   | 1 |   |   |
| 4 |   | 1 |   |   |   |

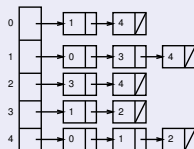
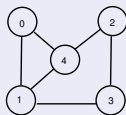
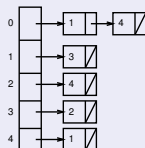
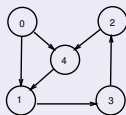


|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 |   | 1 |   |   | 1 |
| 1 | 1 |   |   | 1 | 1 |
| 2 |   |   |   | 1 | 1 |
| 3 |   | 1 | 1 |   |   |
| 4 | 1 | 1 | 1 |   |   |

- ◇ An  $|V| \times |V|$  2D array (matrix), the dimensions representing vertices,  $v_i \in v_0, \dots, v_{n-1}$ , each represented as a bit, if the edge exists (or not)
- ◇ Requires  $\Theta(|V|^2)$  space.

# Graph Representations

## Adjacency List



- ◇ An array of size  $|V|$ , each of which is a linked list.
- ◇ Position  $i$  of the array, stores all edges from vertex  $v_i$ ; each node contains the terminating vertex,  $v_j$ .
- ◇ Requires  $\Theta(|V| + |E|)$  storage.



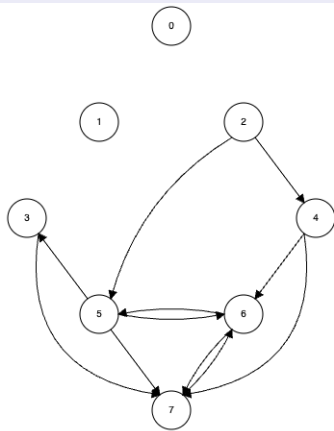
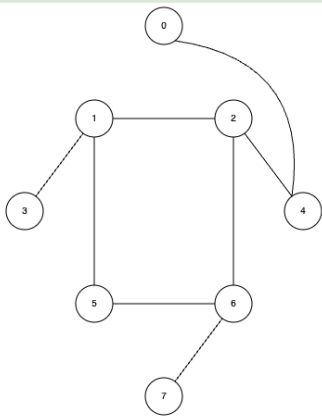
# Graph Representation: Adjacency List vs. Adjacency Matrix

Given a graph of  $V$  vertices and  $E$  edges, what is the complexity of the following graph operations using an adjacency matrix or adjacency list?

- ◇ Does edge  $(v_1, v_2)$  exist?
- ◇ Adding an edge  $(v_1, v_2)$  to the graph.
- ◇ Iterating through all the edges from a vertex  $v_1$
- ◇ List the edges incident on a vertex  $v$
- ◇ Removing an edge  $(v_1, v_2)$  from the graph
- ◇ Removing all edges from  $v_1$

# Problems: Graph Representation

Determine the adjacency matrix and adjacency list representations of the following graphs



# Graph Traversals

- ◇ Goal is to **visit** the nodes of a graph.
- ◇ Need a **start** vertex.
- ◇ **Problems:** graph may be **disconnected**, and contain **cycles**.
- ◇ Use a **mark** bit to identify visited nodes.

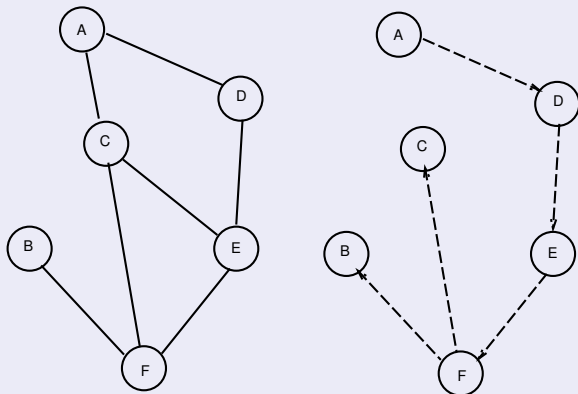
## Algorithm

```
void GraphTraverse(Graph G) {  
    for (v=0; v < G.size(); v++)  
        G.setMark(v, UNVISITED); //Initialize mark bits  
    for (v=0; v < G.size(); v++)  
        if (G.getMark(v) == UNVISITED)  
            doTraverse(G, v);  
}
```

**Note:** Algorithm handles **disconnected graphs**

# Graph Traversals

## Depth First Search (DFS)



- ◇ When a node  $v$  is visited, recursively visit all of its neighbors.
- ◇ Effect is to follow one branch all the way (**as deep as possible**) through and then back up and follow another branch, etc.

# Depth First Search

## Algorithm

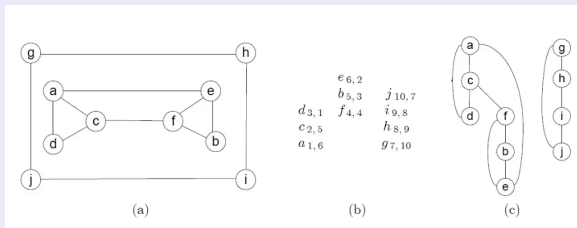
```
// Implements Depth First Search
//Input: Graph  $G = \langle V, E \rangle$ 
//Output: Graph  $G$  with vertices marked with integers
//indicating visit order
for (each vertex  $v \in V$ )
    mark  $v$  as UNVISITED
for (each vertex  $v \in V$ )
    if ( $v == \text{UNVISITED}$ )
        dfs( $v$ )

void dfs( $v$ ) {
    // visits all unmarked vertices connected to  $v$ 
    // and numbers them in order
    mark  $v$  as VISITED
    for (each vertex  $w \in V$  adjacent to  $v$ )
        if ( $w == \text{UNVISITED}$ )
            dfs( $w$ );
}
```

Check out this [link for an interactive demo of DFS](#)

# Depth First Search: Complexity, Properties

## Properties



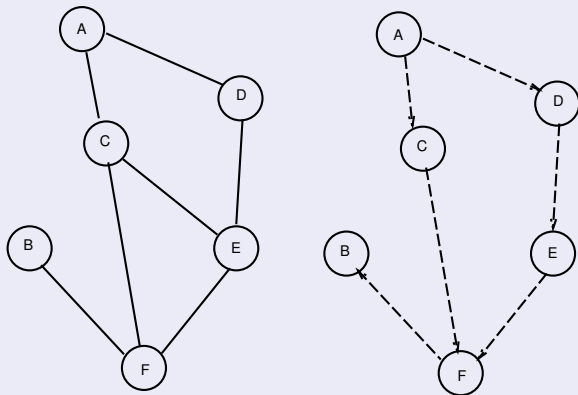
- ◇ DFS is recursive, uses a stack
- ◇ **DFS Forests:**
  - ▷ **Tree edges:** Trees that can be built using the start vertex as the root and adding edges to unvisited vertices.
  - ▷ **Back edges:** Edges that connect back to visited vertices
- ◇ **Application:** Checking Connectivity of a graph, or better, determining all **connected components** of a graph
- ◇ **Application:** Is a graph acyclic? Check for back edges

# Depth First Search: Complexity, Properties

## Complexity

- ◇ Initialization (to mark all vertices as UNVISITED):  $O(V)$
- ◇  $dfs(v)$  is called for each vertex in the graph, which involves visiting all of  $v$ 's edges in its adjacency list, hence  $O(E)$
- ◇ Complexity for adjacency list based graph is  $O(V + E)$
- ◇ Complexity for adjacency matrix based graph is  $O(V^2)$ , as the entire matrix needs to be visited to get to all possible edges

# Breadth First Search(BFS)



- ◇ Examine all vertices connected to a vertex  $v$ , before visiting its children (**level based traversal**)



# Breadth First Search(BFS)

## Algorithm

```
//Implements Breadth First Search  
//Input: Graph  $G = \langle V, E \rangle$   
//Output: Graph  $G$  with vertices marked with integers  
//indicating visit order  
for (each vertex  $v$  in  $V$ )  
    mark  $v$  as UNVISITED  
for (each vertex  $v \in V$ )  
    if ( $v$  is UNVISITED)  
         $bfs(v)$   
  
 $bfs(v)$   
// visits all unvisited vertices connected to  $v$  and  
// marks them using the count value  
mark  $v$  as VISITED, initialize a queue  $Q$   
Enqueue  $v$   
while ( $Q$  is not empty) { // Process each vertex on  $Q$   
    Dequeue element  $v$  from  $Q$   
    for (each vertex  $w \in V$  adjacent to  $v$ )  
        if ( $w$  is UNVISITED) {  
            mark  $w$  as VISITED  
            Enqueue  $w$   
        }  
}
```

# Breadth First Search: Properties, Complexity

## Properties

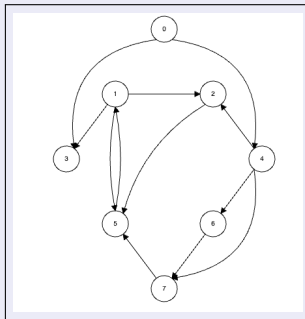
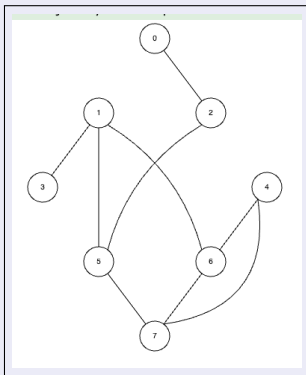
- ◇ Iterative level order traversal, uses a queue
- ◇ For unweighted graphs (or graphs with identical weights for all edges), Dijkstra's shortest path algorithm reduces to BFS
- ◇ BFS Forest: Similar to DFS, has **tree edges**

## Complexity

- ◇ Similar to DFS, the initialization to mark all vertices as UNVISITED costs  $O(V)$
- ◇ At most each node  $v$  is queued once; and visiting each node's edges results in a total amount of work related to the total number of edges  $O(E)$
- ◇ Complexity for an adjacency list based graph is  $O(V + E)$
- ◇ Complexity for an adjacency matrix based graph is  $O(V^2)$

# BFS/DFS Exercises

Perform DFS and BFS traversals on the following graphs and write down the sequence of visited vertices



Additional Practice: Visit [this link](#) and generate graphs

# Graph Basics: Expectations

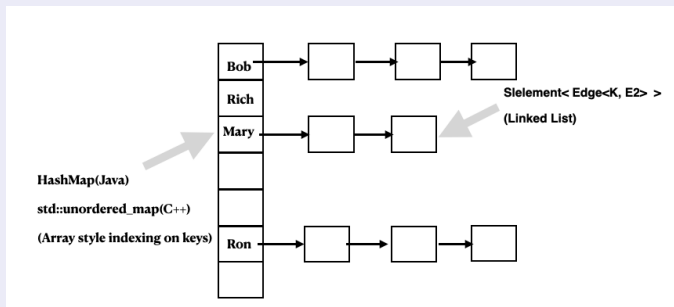
- ◇ Good understanding of the two representations (Adj. matrix, Adj. list) of the graph and what are their tradeoffs, when does it make sense to use each of them?
- ◇ Given a graph, you must be able to draw both of their representations by just looking at a diagram of the vertices and edges.
- ◇ Justify the computational complexity of operations on the graph (see [Slide 9](#))
- ◇ Given a graph, enumerate the nodes visited using BFS and DFS for both directed and undirected graphs (see [Slide 19](#))
- ◇ Justify the complexity of BFS and DFS algorithms.
- ◇ How do you determine the connected components of a graph using BFS or DFS?
- ◇ How do you determine if a graph has cycles? (Look for back edges)

# AdjacencyList based Graphs in BRIDGES

## GraphAdjList Class

- ◇ Java:
  - ◇  $\text{GraphAdjList}\langle K, E1, E2 \rangle$  :  $K$  is key,  $E1$  and  $E2$  are vertex and edge data
  - ◇  $\text{GraphAdjListSimple}\langle K, K, K \rangle$  : Simplified form, with vertex and edge data of the same type as the key
- ◇ C++:
  - ◇  $\text{GraphAdjList}\langle K, E1 = K, E2 = E1 \rangle$  :  $K$  is key,  $E1$  and  $E2$  are vertex and edge data
  - ◇  $\text{GraphAdjList}\langle K \rangle$  is the same as  $\text{GraphAdjList}\langle K, K, K \rangle$
- ◇ Python:
  - ◇  $\text{GraphAdjList}()$

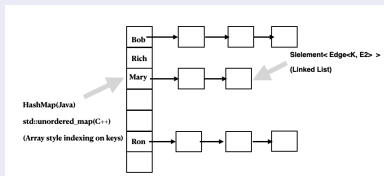
# Bridges GraphAdjList Data Structure



- ◇ Hashmaps (or hash tables) permit array style indexing, extending it to other orderable keys, like strings.
- ◇ Access to the adjacency list in constant time ( $O(1)$ )
- ◇ Linked list uses the BRIDGES SLelement type, and having an Edge type for its generic parameter, holding edge information.
- ◇ Complicated type, but simplified by using **iterators** to access vertices and edges and convenient methods to set visual attributes.

# Bridges GraphAdjList Data Structure

## Accessing Vertices, Edges Using Iterators (Java)

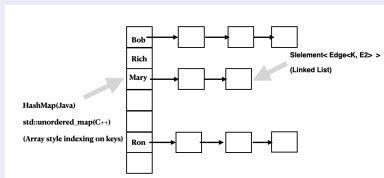


```
GraphAdjListSimple<String> graph =  
    new GraphAdjListSimple<String>();  
for (String actor: graph.getVertices().keySet())  
    System.out.println("\t" + actor);
```

```
GraphAdjListSimple<String> graph =  
    new GraphAdjListSimple<String>();  
for (Edge<String, String> e : graph.outgoingEdgeSetOf("a"))  
    System.out.println("a->" + e.getTo());
```

# Bridges GraphAdjList Data Structure

## Accessing Vertices, Edges Using Iterators (C++)



```
GraphAdjList<string> graph;  
for (auto actor : graph.keySet())  
    cout << vert << "\n";
```

```
GraphAdjList<string> graph;  
for (auto& edge : graph.outgoingEdgeSetOf("kevin_bacon"))  
    cout << edge.from() << "—" << edge.to() << "\n";
```